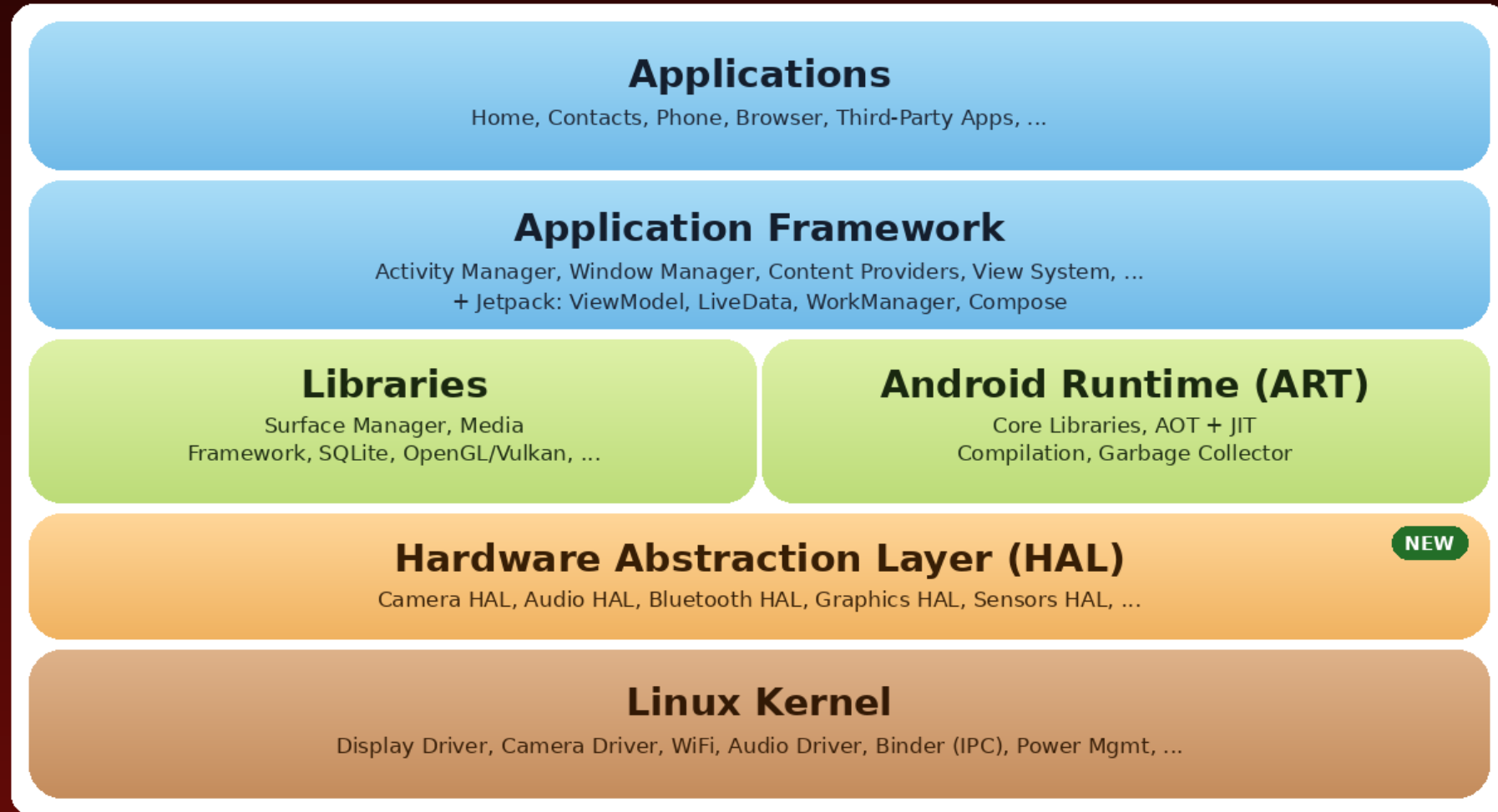


# ANDROID

Dr.K.Sathiyamurthy,  
Professor, DCSE,PTU

# ANDROID ARCHITECTURE

Updated — Android 8.0+ (current)



*Corrected from the classic diagram: ART (Android Runtime) replaced Dalvik VM in Android 5.0 (2014) — Dalvik is no longer used on any modern device. Hardware Abstraction Layer (HAL) added as its own tier, formalized by Project Treble in Android 8.0 (2017) to speed up OS updates across manufacturers.*

# ART VS DVM IN ANDROID

| Feature             | DVM (Dalvik Virtual Machine)                                                                | ART (Android Runtime)                                                                                     |
|---------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Introduction        | Used in Android $\leq$ 4.4 (KitKat)                                                         | Default from Android 5.0 (Lollipop) onwards                                                               |
| Execution Type      | Just-In-Time (JIT) (compilation – code is compiled into machine code at runtime, as needed) | Ahead-Of-Time (AOT) –(compilation – code is compiled into native machine code when the app is installed.) |
| Performance         | Slower at runtime(since compilation happens during execution.)                              | Faster runtime execution (Faster execution because code is already compiled before running.)              |
| Memory Usage        | Lower at install, higher during run                                                         | Higher at install, lower during run                                                                       |
| Battery Efficiency  | Less efficient (more CPU cycles)                                                            | More efficient (reduced CPU cycles)                                                                       |
| App Launch Time     | Slower                                                                                      | Faster                                                                                                    |
| Storage Requirement | Less storage (bytecode only)                                                                | More storage (compiled OAT files)                                                                         |
| Garbage Collection  | Older GC, UI lags                                                                           | Improved GC, smoother UI                                                                                  |

# CAMERA CAPTURE — LAYER-BY-LAYER ACTIVITY FLOW

Tracing one user action down through the Android stack to hardware, and back up

▼ REQUEST — flowing down

▲ RESPONSE — flowing back up

Applications

1

User taps the Capture button on screen

10

App updates its UI — photo thumbnail is shown to the user

Application Framework

2

Activity's click handler calls the Camera API

9

Content Provider saves the file so other apps (e.g. Gallery) can access it

Libraries & Android Runtime (ART)

3

ART executes the call; Media Framework reads the capture pipeline

8

Media Framework compresses it; ART runs the app's save logic

Hardware Abstraction Layer (HAL)

4

Camera HAL translates the request into this device's specific instructions

7

Camera HAL formats the raw data into a standard image format

Linux Kernel

5

Camera Driver signals the physical sensor to capture a frame

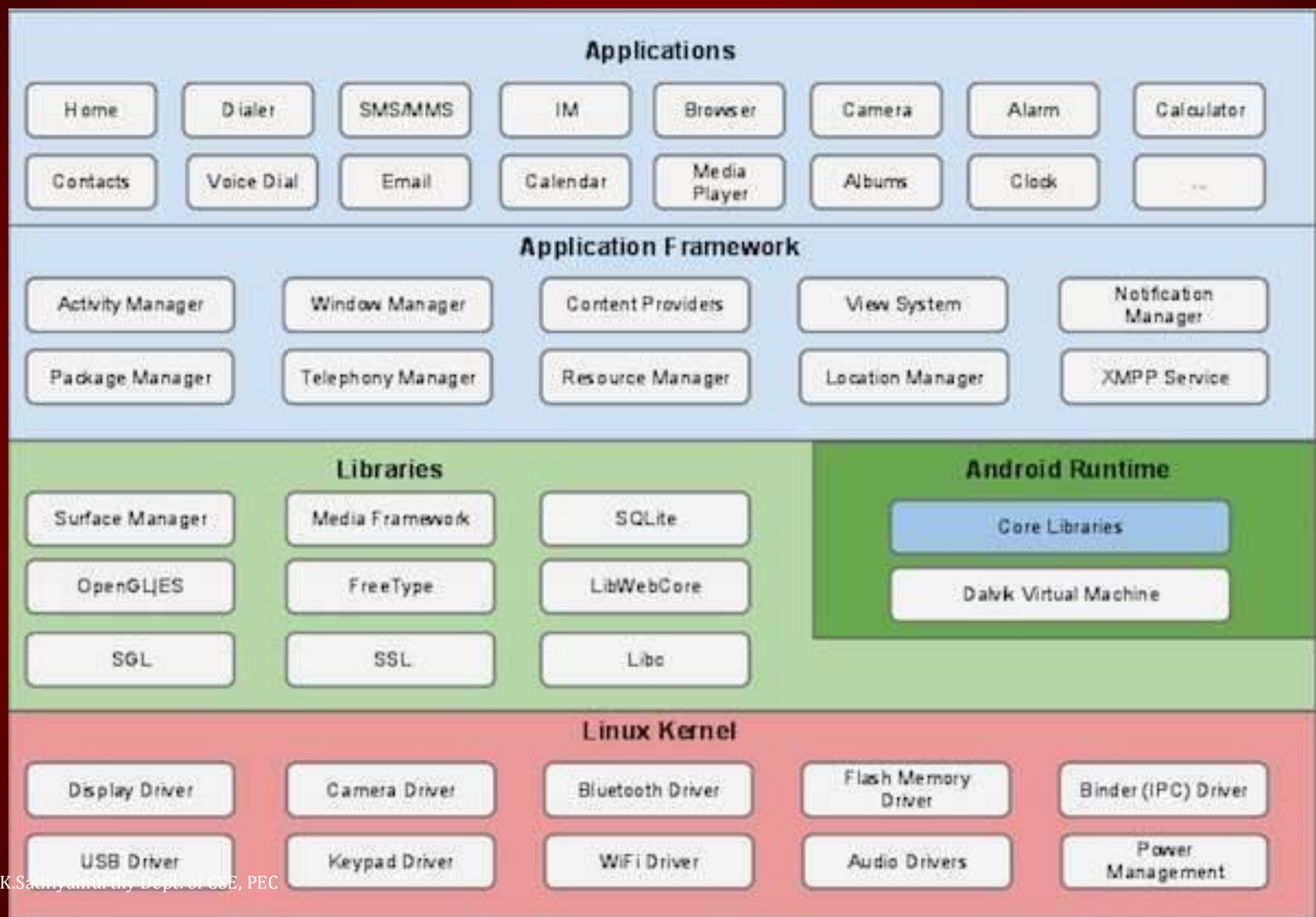
6

Driver reads the raw captured data back from the sensor

Physical Hardware

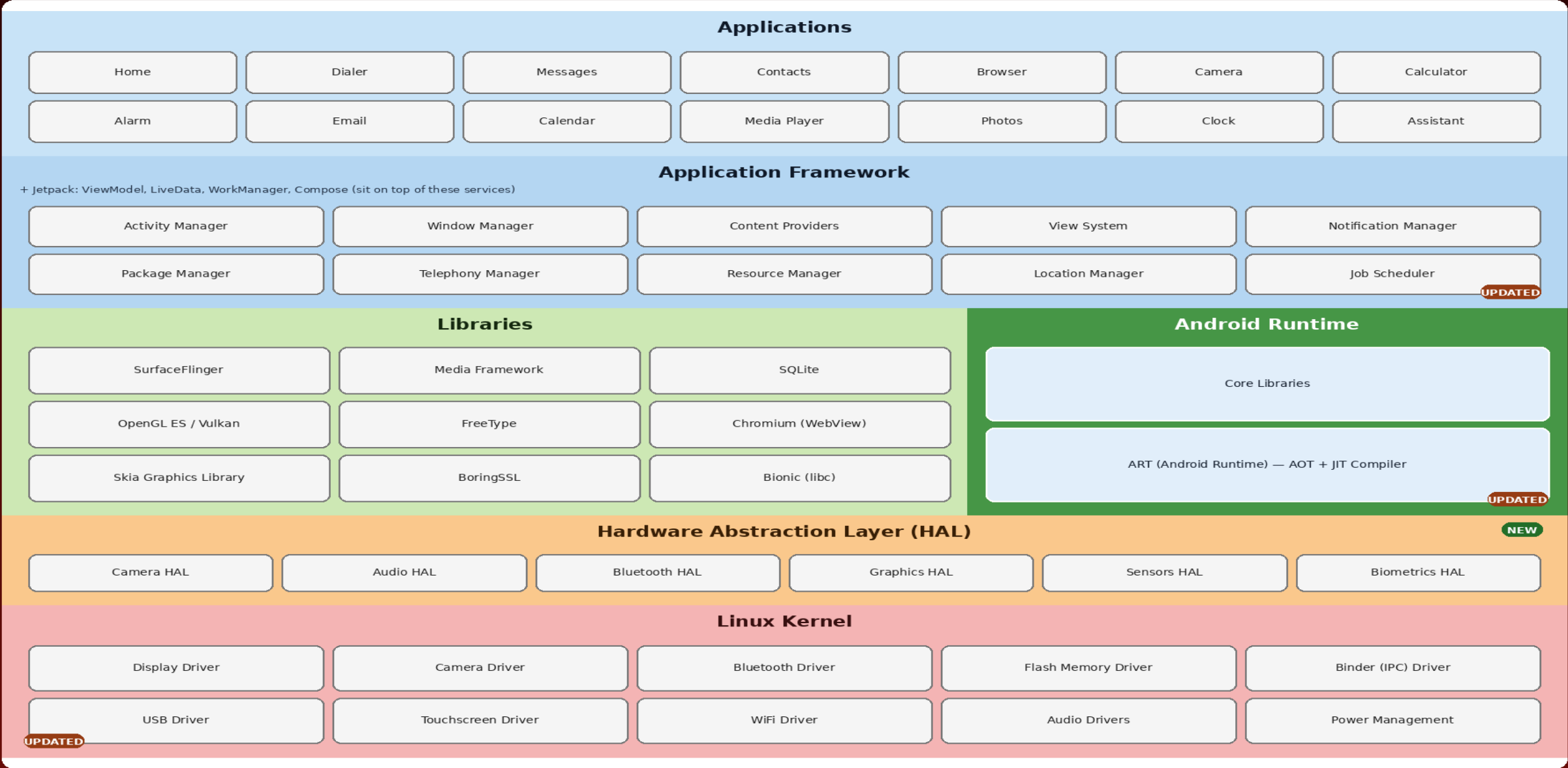
Camera Image Sensor — captures raw photo data

Every user action makes this same round trip: down through five layers to the hardware, then straight back up — no layer needs to know how any other layer does its job.



# ANDROID ARCHITECTURE — FULL COMPONENT VIEW

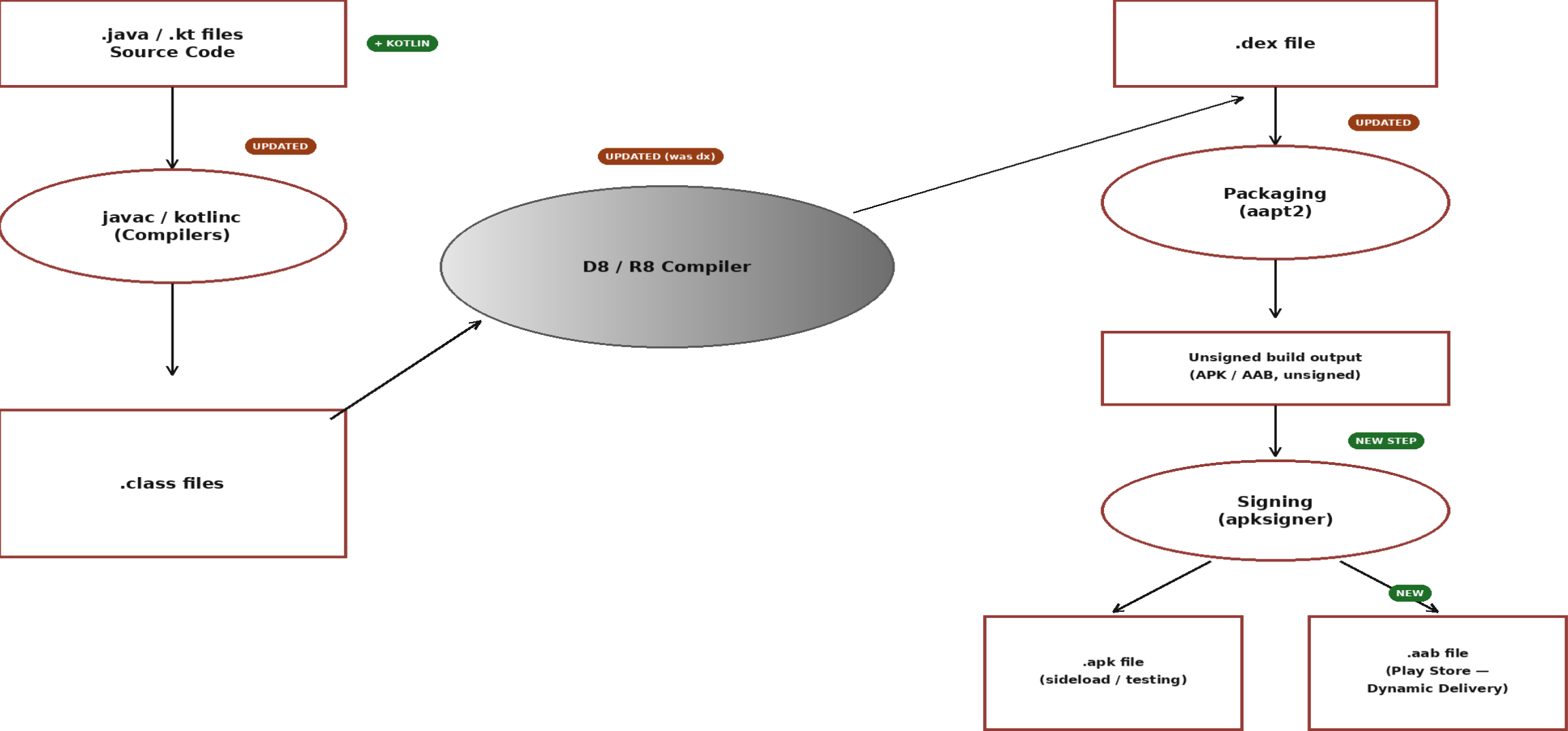
Updated for current Android (ART runtime, Chromium WebView, HAL layer per Project Treble)



Corrections from the classic diagram: ART replaced Dalvik VM (Android 5.0, 2014) · LibWebCore replaced by Chromium WebView (Android 4.4, 2013)  
Keypad Driver replaced by Touchscreen Driver (touch-first devices) · XMPP Service replaced by Job Scheduler (background task scheduling, API 21+)  
Hardware Abstraction Layer (HAL) added as its own tier, formalized by Project Treble (Android 8.0, 2017) — was previously folded into the Kernel layer.

# ANDROID BUILD PROCESS — CURRENT TOOLCHAIN

Updated: D8/R8 replace dx, aapt2 replaces aapt, Kotlin added, App Bundle + signing added



dx → D8/R8 (default dex compiler + shrinker since 2018–2019) · aapt → aapt2 (default resource compiler since 2017)  
Kotlin source added alongside Java — Kotlin has been Google's preferred Android language since 2019, compiling to the same .class format.  
Signing and the Android App Bundle (.aab) added — Google Play has required .aab for new app publishing since August 2021.

# ANDROID BUILD PROCESS — ENGINEERING MARVELS

*Five design decisions that make the modern toolchain remarkable*

---

## **Kotlin + Java -> One Bytecode Format**

1

kotlinc and javac both compile down to the identical .class format, so the two languages call each other directly with zero glue code — how Kotlin was adopted without splitting the ecosystem.

---

## **D8/R8 — Four Jobs in a Single Pass**

2

One tool converts stack-based JVM bytecode to register-based Dalvik bytecode, shrinks unused code app-and-library-wide, obfuscates names, and optimizes — replacing three separate legacy tools.

---

## **AAPT2 — Resources Compiled to Integers**

3

Every string/layout/drawable reference becomes a compile-time integer ID (R.id...), avoiding costly runtime string lookups on constrained mobile hardware.

---

## **AAB + Dynamic Delivery**

4

One upload; Google Play's servers generate a custom-fit APK per device (density, CPU architecture, language) at download time — solving device fragmentation centrally instead of per-developer.

---

## **Modern APK Signing (v2+)**

5

Signs the whole archive as one block instead of hashing every file individually, making install-time verification nearly instant even for huge, multi-dex apps.

---

*The unifying idea: dx -> D8/R8, aapt -> aapt2, JAR signing -> modern signing — every swap happened underneath a stable build contract, with zero source-code changes required from developers.*



Alpha  
Android 1.0



Beta  
Android 1.1



Cupcake  
Android 1.5



Donut  
Android 1.6



Eclair  
Android 2.02.1



Froyo  
Android 2.2



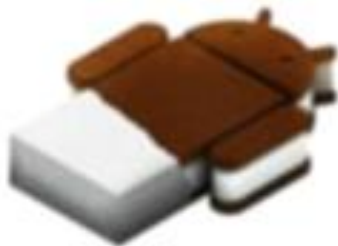
Gingerbread  
Android 2.3



# ANDROID VERSIONS



Honeycomb  
Android 3.0



Ice Cream Sandwich  
Android 4.0



Jelly Bean  
Android 4.1



KitKat  
Android 4.4



Lollipop  
Android 5.0



Marshmallow  
Android 6.0



**Alpha**

Android 1.0



**Beta**

Android 1.1



**Cupcake**

Android 1.5



**Donut**

Android 1.6



**Eclair**

Android 2.0-2.1



**Froyo**

Android 2.2



**Gingerbread**

Android 2.3



**Honeycomb**

Android 3.0



**Ice Cream Sandwich**

Android 4.0



**Jelly Bean**

Android 4.1-4.3



**KitKat**

Android 4.4



**Lollipop**

Android 5.0-5.1



**Marshmallow**

Android 6.0



# ANDROID VERSIONS



**Nougat**

Android 7.0-7.1



**Oreo**

Android 8.0-8.1



**Pie**

Android 9

*Android 10 and later - numbered releases only; dessert names used internally only*



**Android 10**

*Quince Tart*



**Android 11**

*Red Velvet Cake*



**Android 12 / 12L**

*Snow Cone*



**Android 13**

*Tiramisu*



**Android 14**

*Upside Down Cake*



**Android 15**

*Vanilla Ice Cream*



**Android 16**

*Baklava*



**Android 17**

*Cinnamon Bun*

Updated through Android 17 (Cinnamon Bun) - stable release June 2026



# ANDROID VERSION FEATURES

Part 1: The Dessert-Name Era - Android 1.0 to 9.0

| Ver.    | Name               | Year    | Key Feature(s)                                                              |
|---------|--------------------|---------|-----------------------------------------------------------------------------|
| 1.0     | Alpha              | 2008    | First public Android release; basic phone OS, Android Market debut          |
| 1.1     | Beta               | 2009    | Bug fixes and minor API updates over Alpha                                  |
| 1.5     | Cupcake            | 2009    | First on-screen keyboard, home-screen widgets, video recording              |
| 1.6     | Donut              | 2009    | Support for varied screen sizes/resolutions, CDMA networks                  |
| 2.0-2.1 | Eclair             | 2009    | Google Maps Navigation, live wallpapers, speech-to-text                     |
| 2.2     | Froyo              | 2010    | JIT compilation for speed, Wi-Fi hotspot tethering, Flash support           |
| 2.3     | Gingerbread        | 2010    | NFC support, front-facing camera support, improved copy-paste               |
| 3.0     | Honeycomb          | 2011    | Tablet-only release, holographic UI, no physical buttons required           |
| 4.0     | Ice Cream Sandwich | 2011    | Unified phone/tablet UI, face unlock, data usage controls                   |
| 4.1-4.3 | Jelly Bean         | 2012-13 | Project Butter (60fps smoothness), Google Now, multi-user support           |
| 4.4     | KitKat             | 2013    | Optimized for low-RAM devices, immersive mode, "OK Google" hotword          |
| 5.0-5.1 | Lollipop           | 2014-15 | Material Design introduced, ART runtime replaces Dalvik, multi-SIM          |
| 6.0     | Marshmallow        | 2015    | Runtime app permissions, Doze battery mode, fingerprint API, USB-C          |
| 7.0-7.1 | Nougat             | 2016    | Split-screen multitasking, Vulkan graphics API, Daydream VR support         |
| 8.0-8.1 | Oreo               | 2017    | Project Treble (modular HAL), Picture-in-Picture, notification channels     |
| 9       | Pie                | 2018    | Gesture navigation, ML-based adaptive battery/brightness, Digital Wellbeing |



# ANDROID VERSION FEATURES

Part 2: The Numbered Era - Android 10 to 17 (codenames are internal only)

| Ver.     | Codename*         | Year | Key Feature(s)                                                                   |
|----------|-------------------|------|----------------------------------------------------------------------------------|
| 10       | Quince Tart       | 2019 | System-wide dark theme, scoped storage, full gesture navigation                  |
| 11       | Red Velvet Cake   | 2020 | Conversation notifications & chat bubbles, one-time permissions                  |
| 12 / 12L | Snow Cone         | 2021 | Material You dynamic theming, privacy dashboard; 12L for foldables/tablets       |
| 13       | Tiramisu          | 2022 | Per-app language settings, themed app icons, granular media permissions          |
| 14       | Upside Down Cake  | 2023 | Lock-screen customization, improved large-screen/foldable support                |
| 15       | Vanilla Ice Cream | 2024 | Privacy sandbox continuation, battery health tools, partial screen sharing       |
| 16       | Baklava           | 2025 | Live Updates notifications, desktop windowing for tablets, Notification Cooldown |
| 17       | Cinnamon Bun      | 2026 | Stricter background-audio rules, per-app password lock, VVC hardware playback    |

# Android Components

ACTIVITY

SERVICE

BROADCAST  
RECEIVER

CONTENT PROVIDER

# What Is an Activity?

- 1 Each distinct screen in an app is typically backed by one Activity.
- 2 It's a visual representation of part of an application.
- 3 Whatever the user interacts with on that screen is made possible by the Activity behind it.
- 4 An application can contain multiple Activities.

# INTENT

- Intention to perform action
- Contains a motive to trigger an event or to perform an action in the android OS or in any application
- Basic functionality of this intent is to start an activity or launch an application



Start an activity



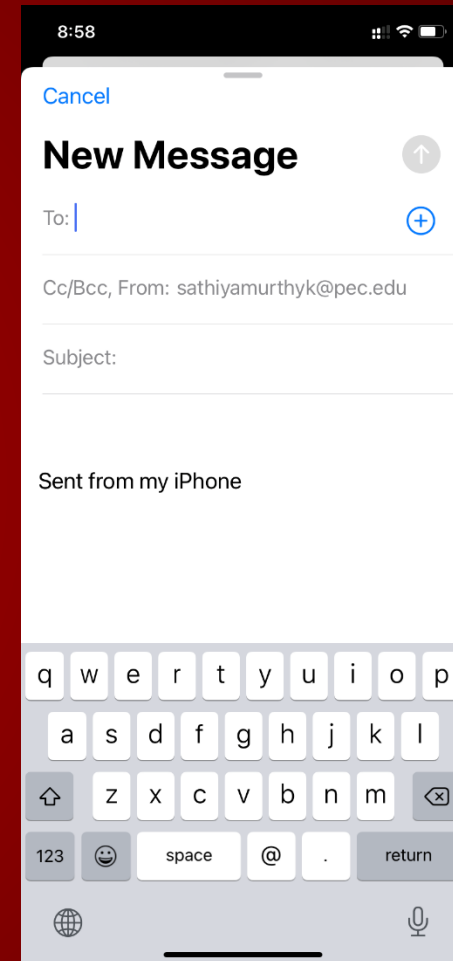
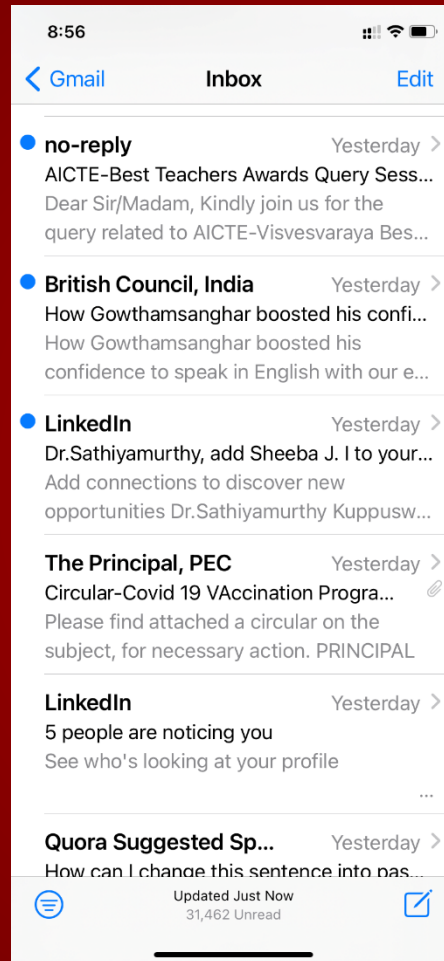
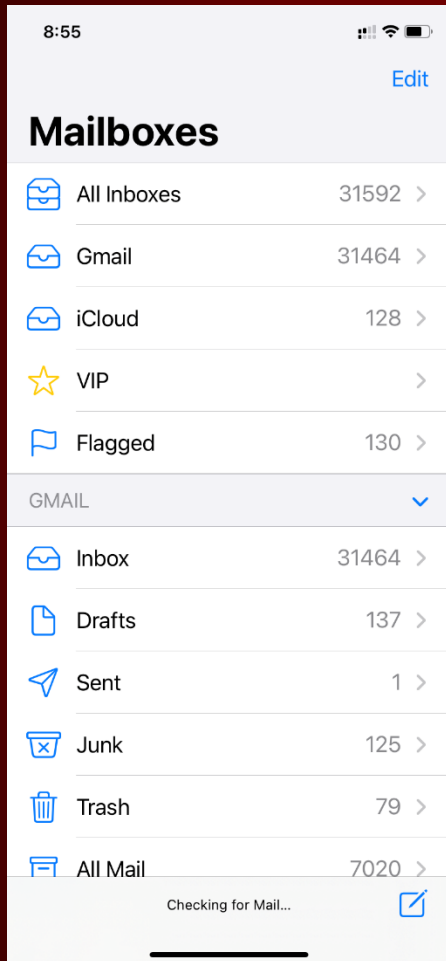
Start a service



Deliver a Broadcast for  
Broadcast receiver

INTENT

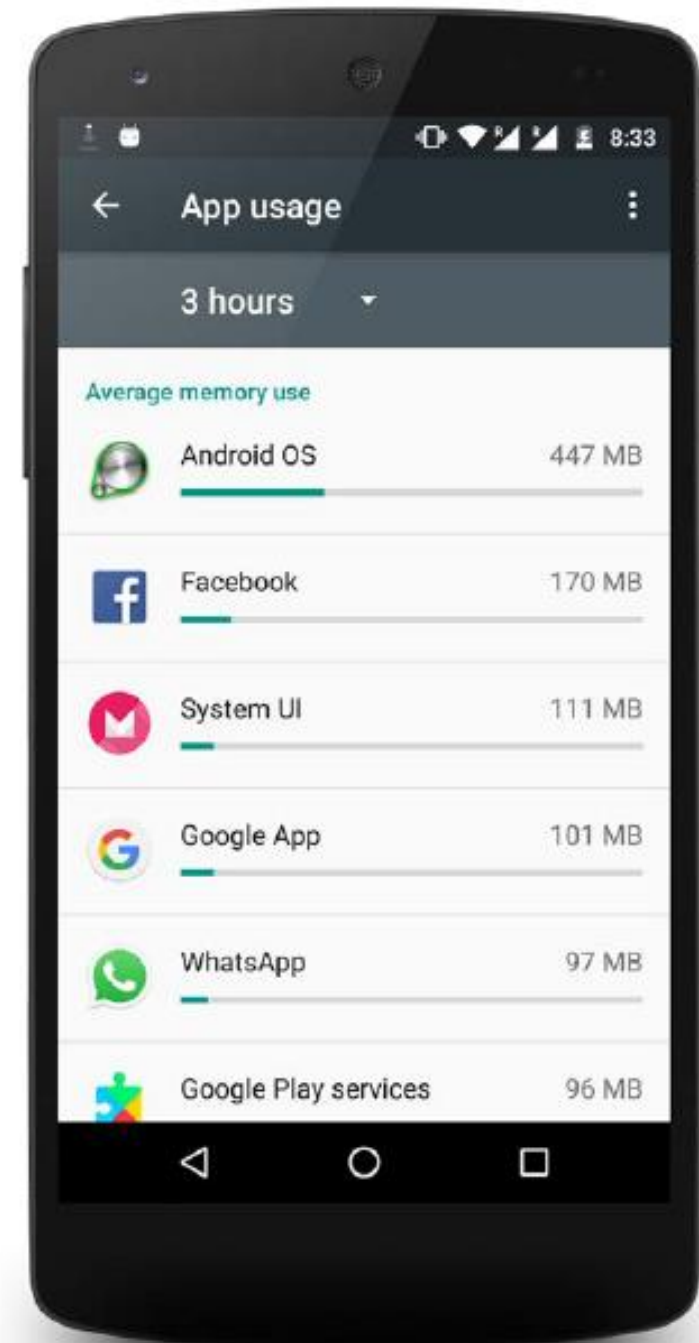
# ACTIVITY



\* Migration from 1 activity to another activity is possible with the help of special class Intent

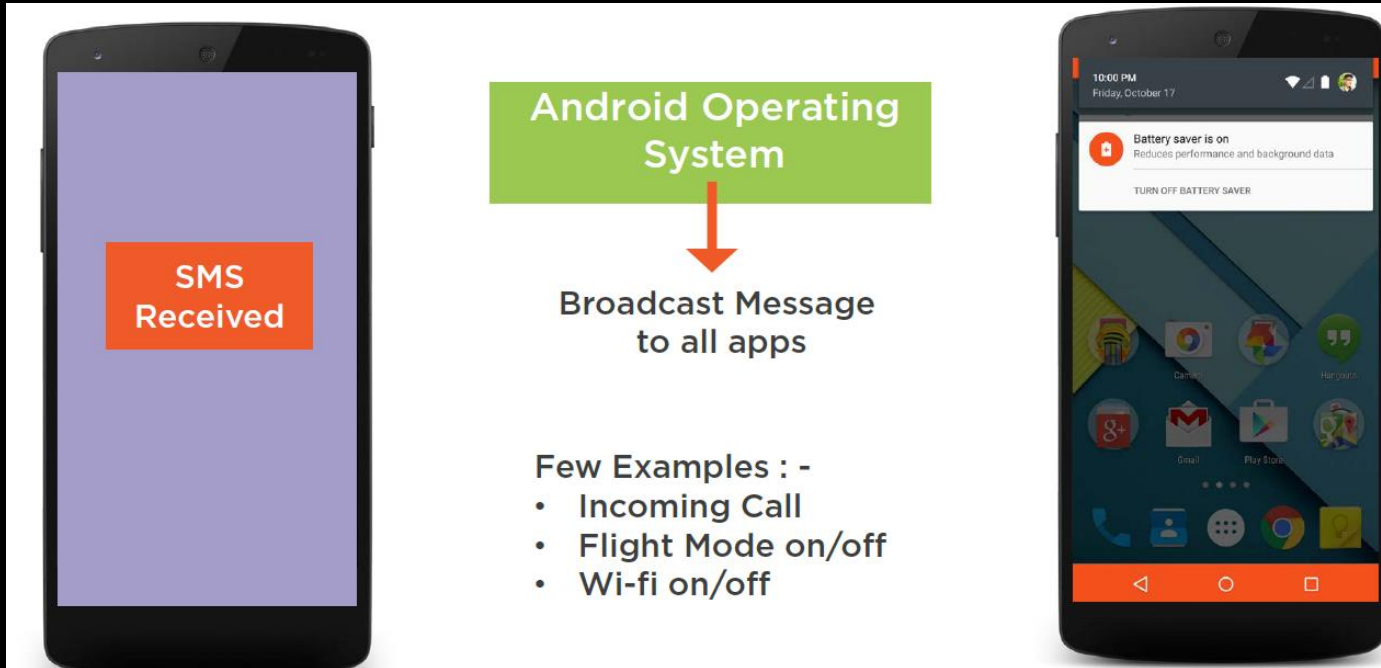
# SERVICE

- ❑ Runs in Background
- ❑ No UI
- ❑ Can be started by activity
- ❑ Can be started by Broadcast receiver
- ❑ Examples
  - ❑ Playing music
  - ❑ Downloading or uploading an app

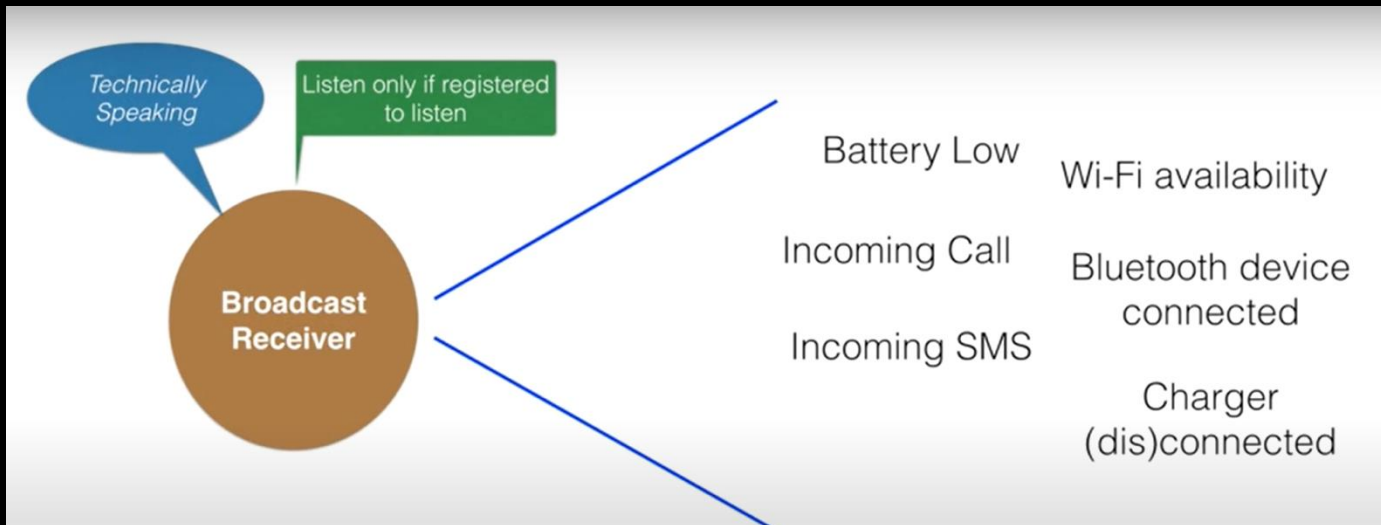


# BROADCAST RECEIVER

Whenever there is some kind of disturbances then AOS broadcast messages to all applications

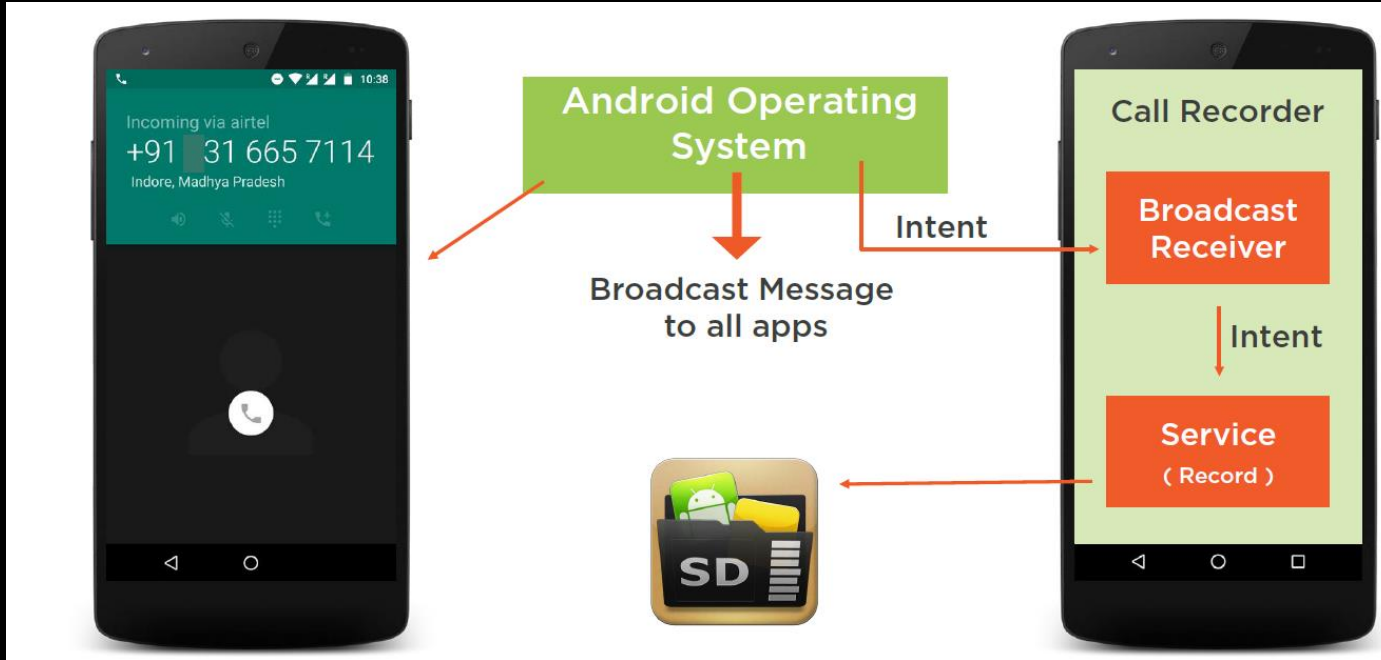


# BROADCAST RECEIVER



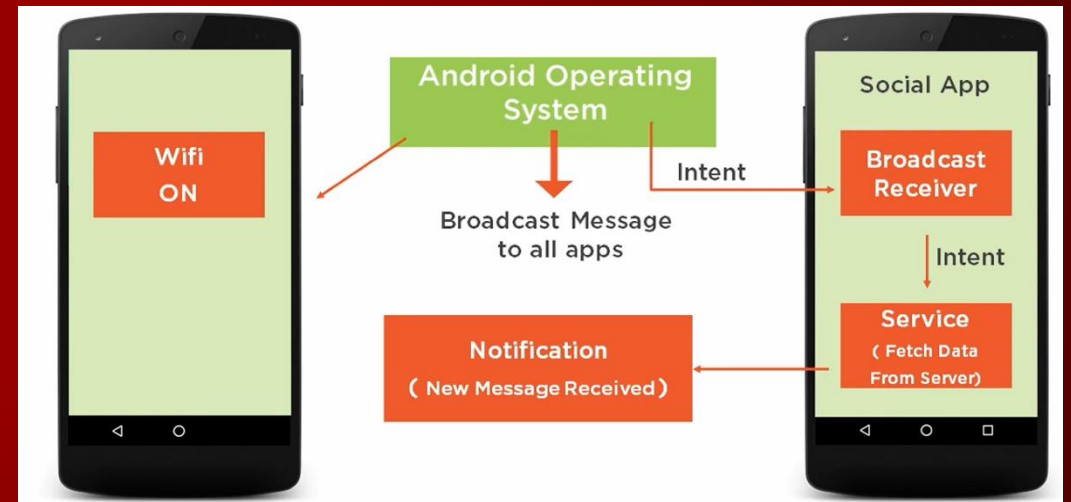
# BROADCAST RECEIVER

Example – call recorder app registered for call receiver

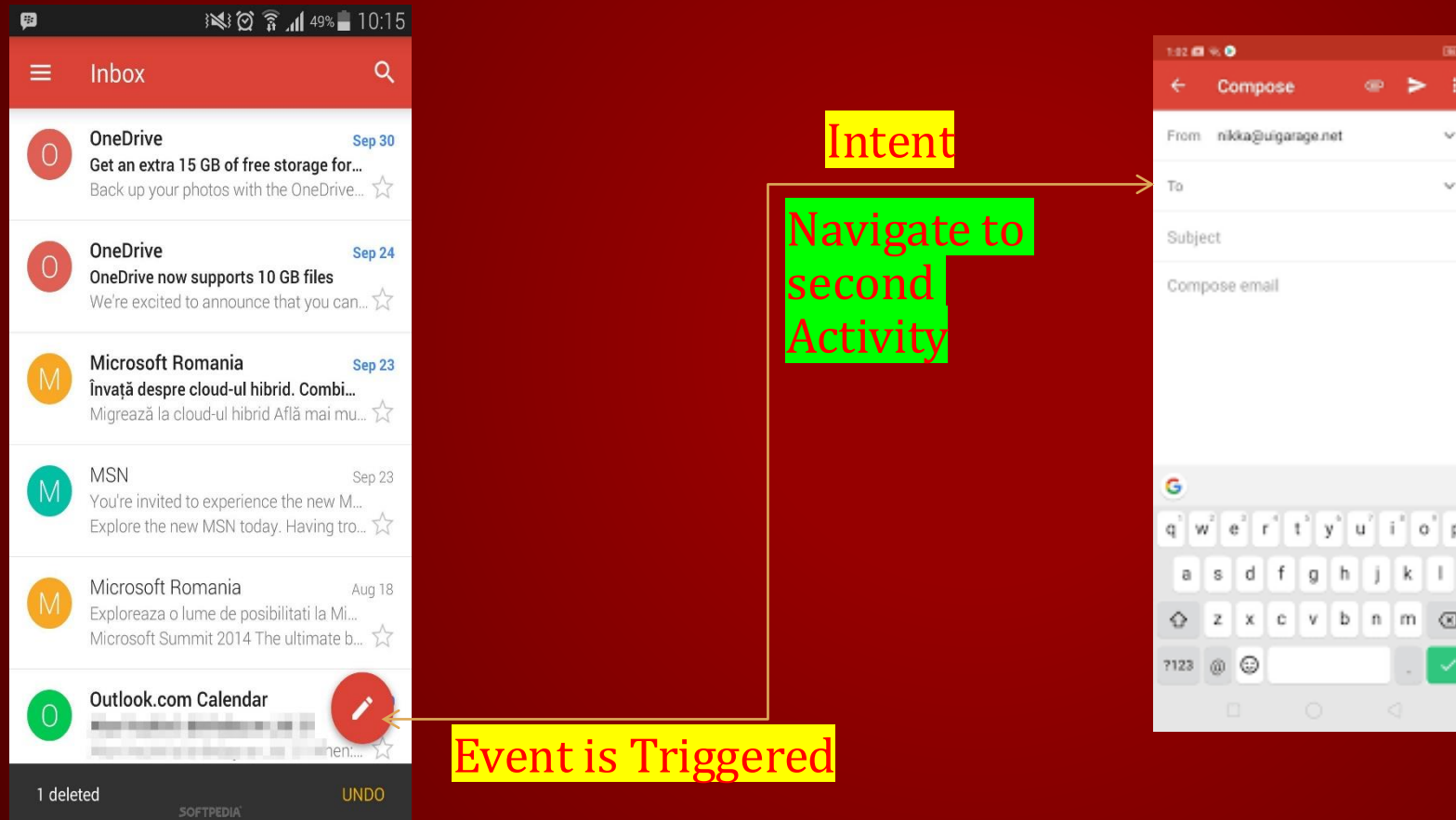


# BROADCAST RECEIVER

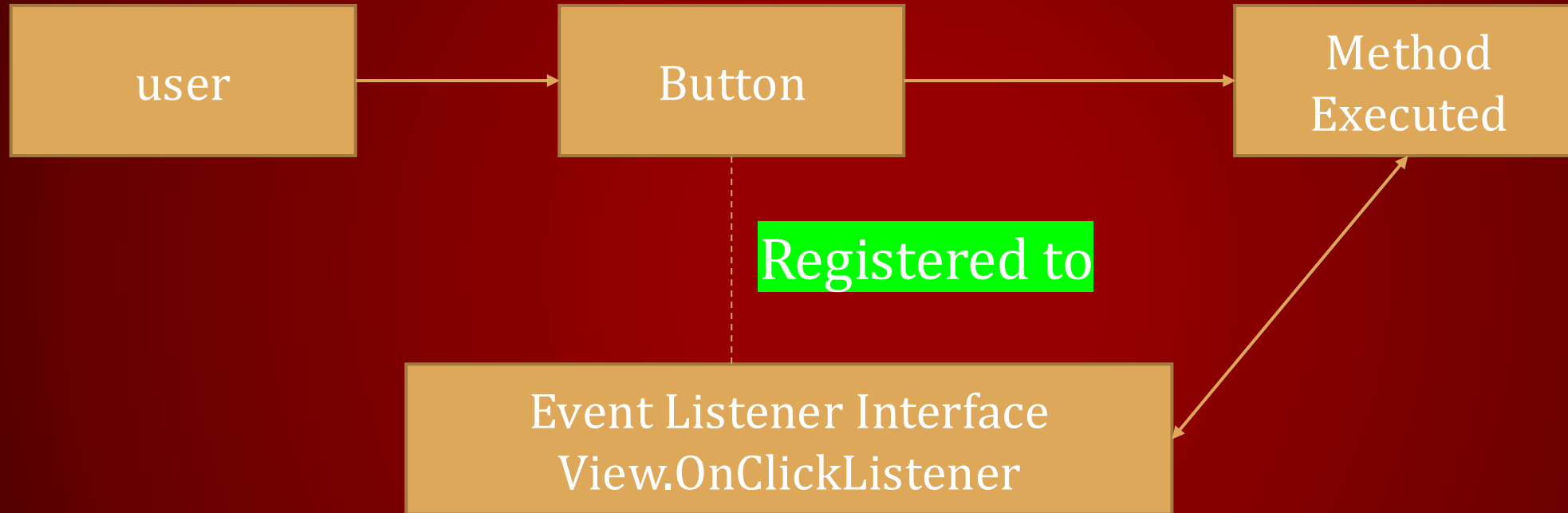
- Example for social media data update with WIFI on



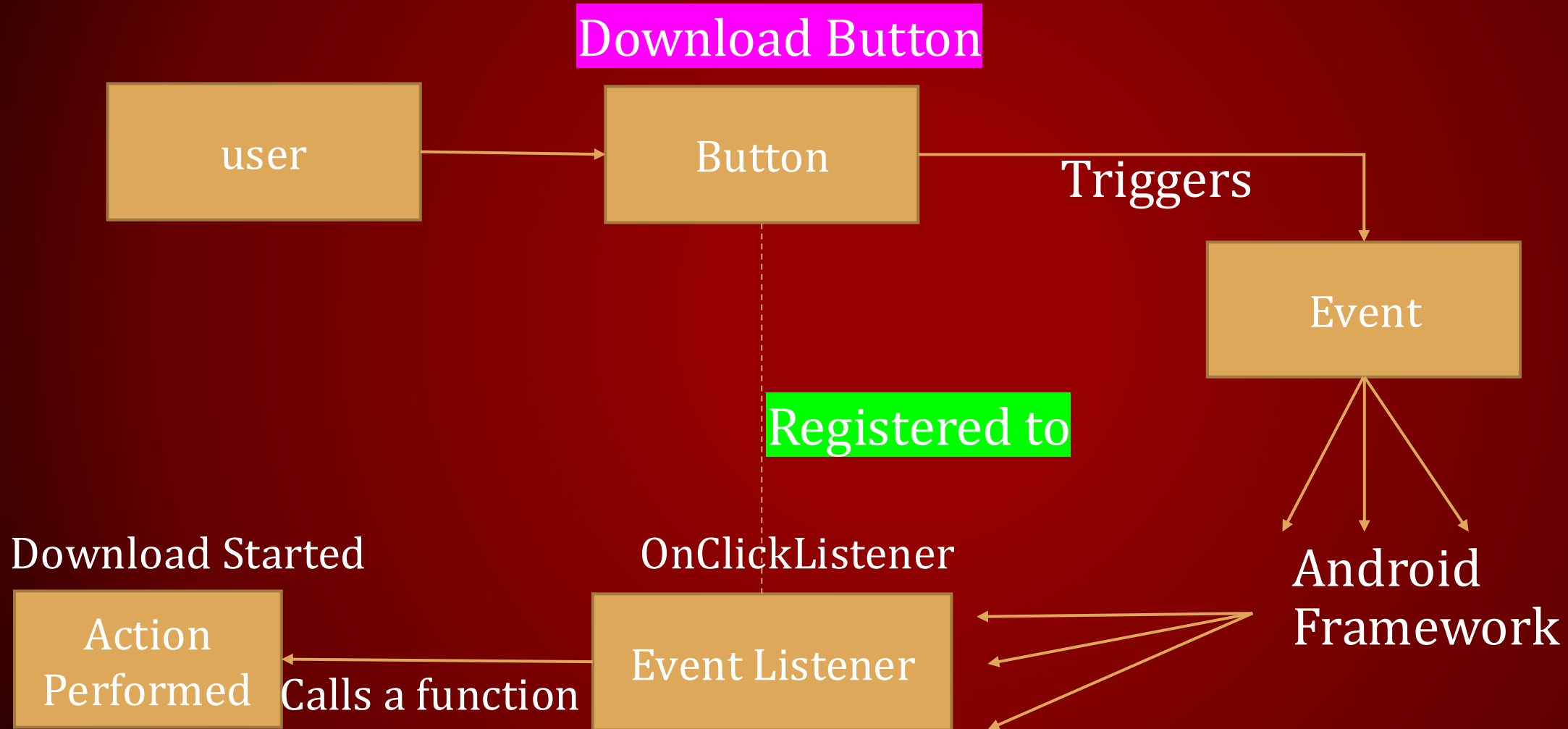
# Using Activities to listen to Events



# Event Handling



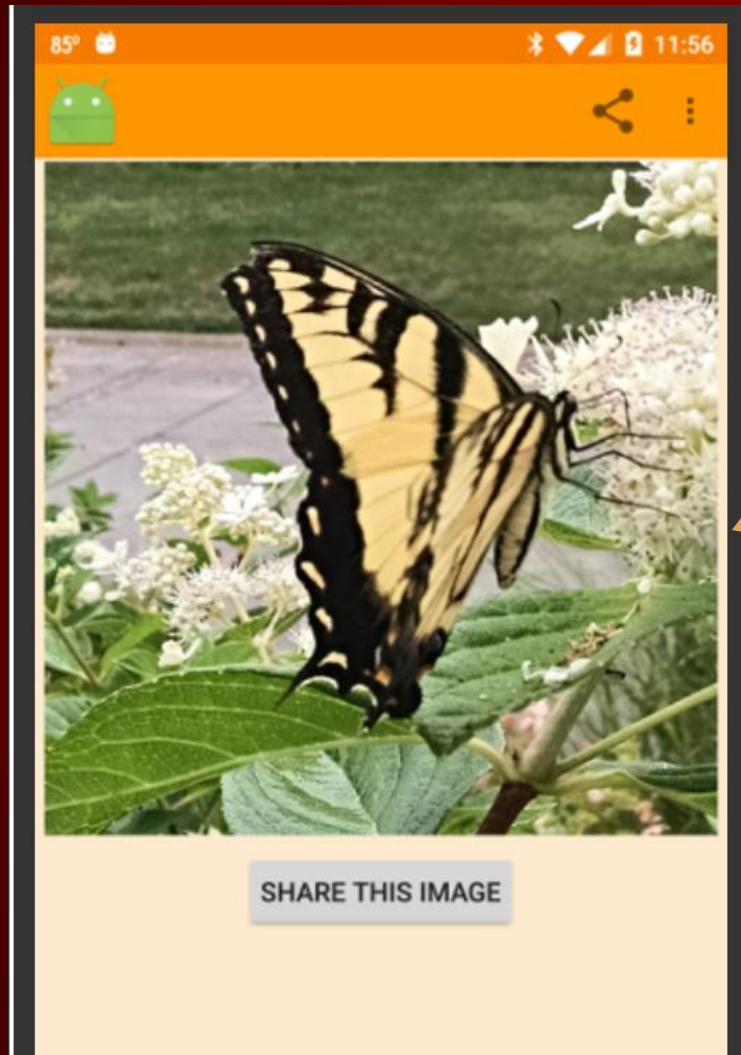
# Event Handling: Downloading Example



# EVENT HANDLING DEMO

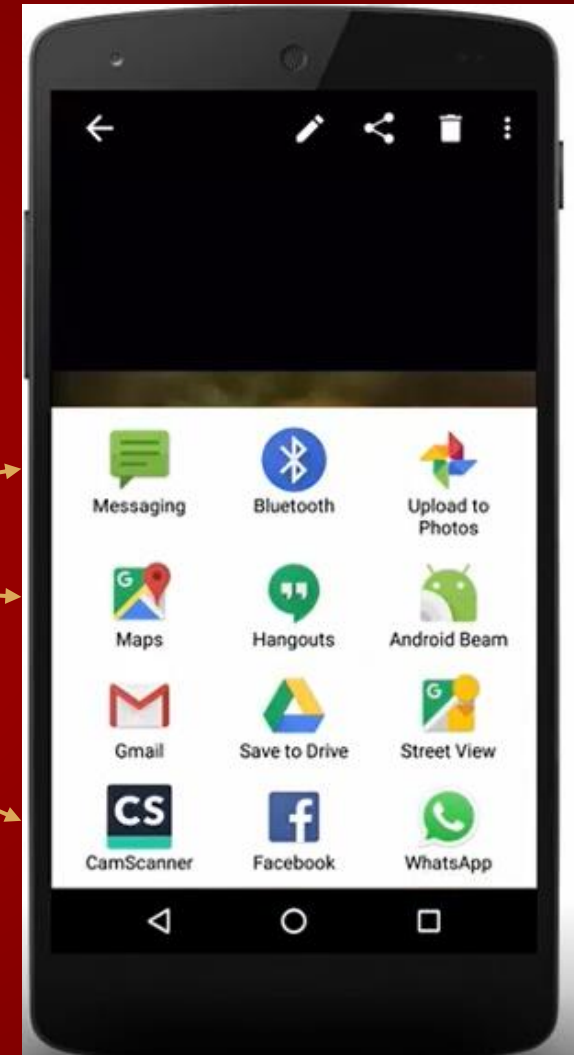
- ❑ 3 Ways of Event Handling
- ❑ Event Handling in Activity for other views
  - ❑ Image touched/clicked
    - ❑ onTouchListener/OnClickListener
  - ❑ Checkbox Checked
    - ❑ OnCheckedChangeListener
  - ❑ RadioButton selected
    - ❑ OnCheckedChangeListener(RadioGroup)

# PASSING DATA ACROSS ACTIVITIES – IMPLICIT INTENT



**IMPLICIT INTENT**

**Sharing Data  
Across different  
App Activities**



## **AppCompatActivity class:**

- ☐ If material design implementation is needed
- ☐ Target android devices below API 21

## **FragmentActivity class:**

- ☐ If nested fragments needed
- ☐ If design features don't require material design for lower API levels

## **Activity class:**

- ☐ None of the above needed

❖ All the features above are essential for professional applications

# SHARING DATA BETWEEN ACTIVITIES

- ❑ Explicit Intent
  - ❑ Target Activity is known
- ❑ Implicit Intent
  - ❑ Target Activity is not known

# Sharing Data Between Activities

## Explicit Intent

### Target Activity is known

You directly specify which Activity (class) should start next — for example, MainActivity to SecondActivity.

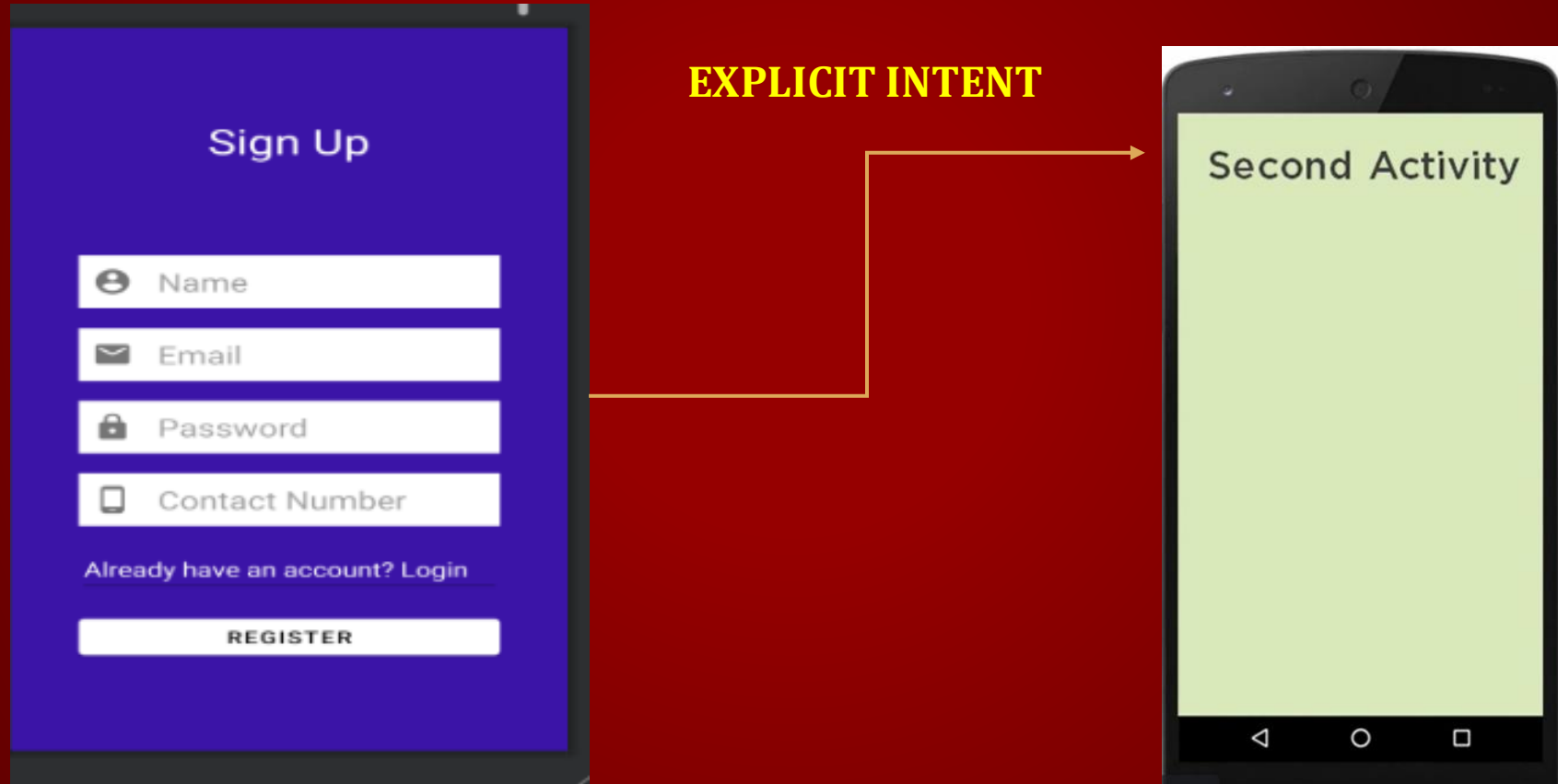
vs

## Implicit Intent

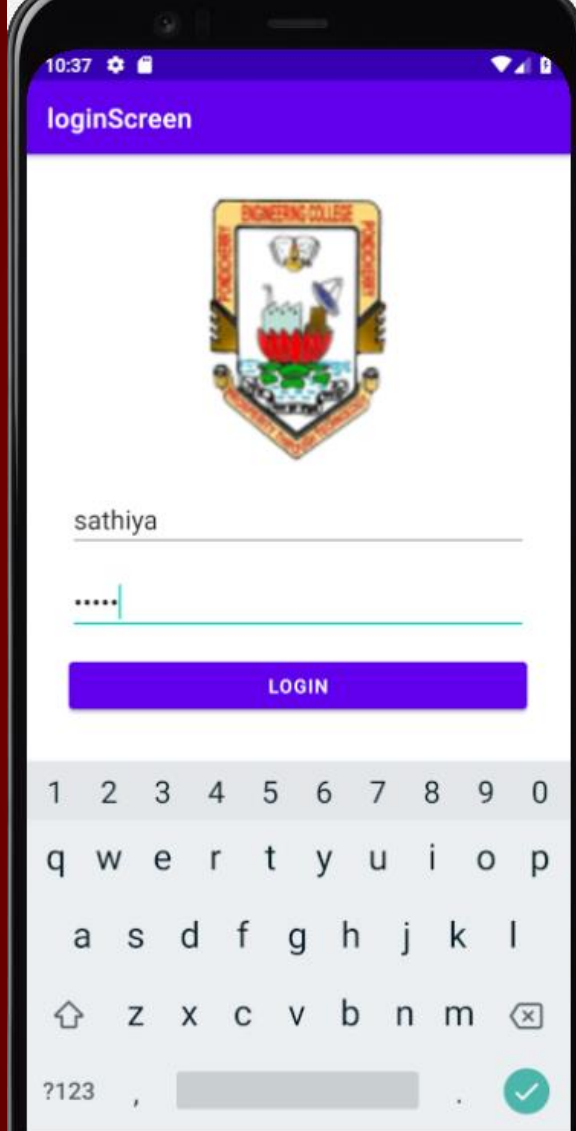
### Target Activity is not fixed

You specify an action instead of an app. Android matches it against apps that can handle that action — and lets the user choose if more than one qualifies.

# PASSING DATA ACROSS ACTIVITIES – EXPLICIT INTENT

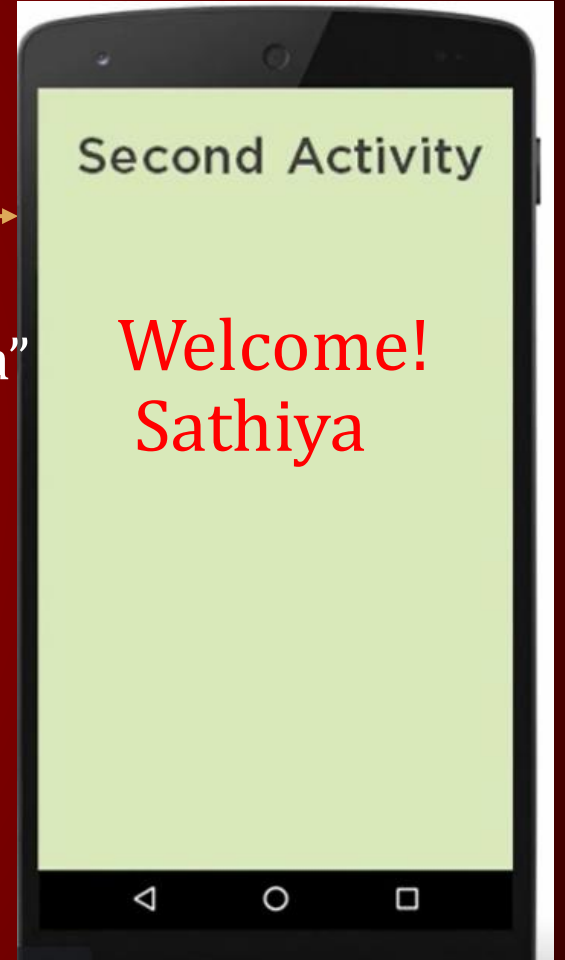


# PASSING DATA ACROSS ACTIVITIES – EXPLICIT INTENT

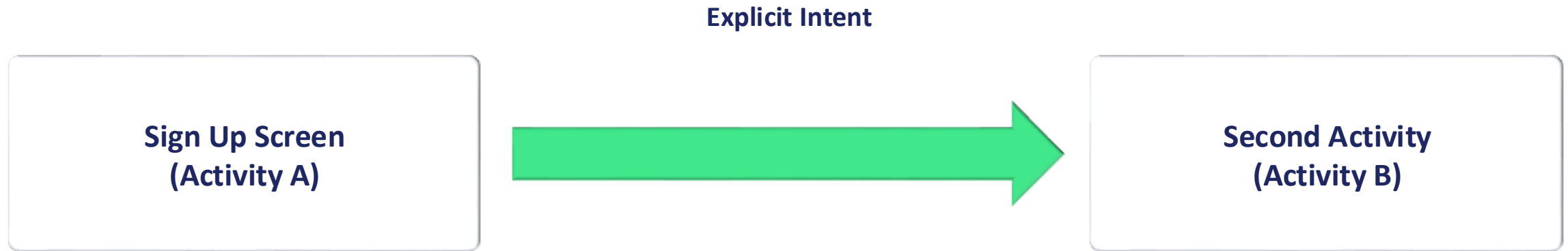


**EXPLICIT INTENT**

Key : Value  
"name": "sathiya"



# Explicit Intent — Concept



The class you want to open is stated directly in code — there is no ambiguity about which screen opens next.

# Explicit Intent — Passing Data

Key : Value

## Login Screen

Enter name: sathiya



`"name" : "sathiya"`

## Welcome Screen

Welcome, Sathiya!

The value is attached to the Intent as a key/value pair and read back by name in the receiving Activity.

# EXPLICIT INTENT

Passing Data to another Activity

- Using Bundle directly
  - Bundle stores the value in a key, value pair
- Using Bundle indirectly

# Explicit Intent — Two Ways to Attach Data

1

## Using Bundle Directly

You create a Bundle object yourself, add key/value pairs to it, then attach the whole Bundle to the Intent.

2

## Using Bundle Indirectly

You call `intent.putExtra()` directly. Android creates and manages the Bundle internally for you, behind the scenes.

# Explicit Intent — Code Comparison

## Bundle — Directly

```
Intent intent = new Intent(...);  
Bundle b = new Bundle();  
b.putInt("key1", value1);  
b.putString("key2", value2);  
intent.putExtras(b);  
startActivity(intent);
```

## Bundle — Indirectly (putExtra)

```
Intent intent = new Intent(...);  
intent.putExtra("key1", value1);  
intent.putExtra("key2", value2);  
startActivity(intent);
```

# Implicit Intent — Example

Photo App — tap “Share this image”



Implicit Intent (ACTION\_SEND)



Messaging



Gmail



WhatsApp



Bluetooth



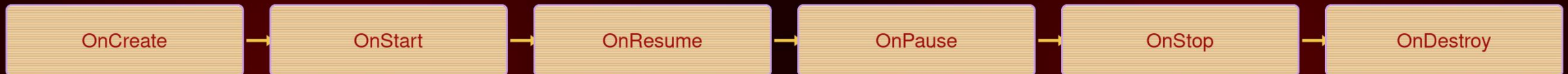
Drive

You never named a specific app in code — Android found every installed app registered to handle ACTION\_SEND and let the user pick.

# Introduction to the Activity Lifecycle

- 1 Every Activity moves through a defined set of states — from the moment it is created to the moment it is destroyed.
- 2 The Android system, not the developer, decides when these transitions happen — a call, a rotation, or switching apps can trigger one at any time.
- 3 Each state change fires a callback method (onCreate, onStart, onResume...) — the developer's one chance to react at exactly the right moment.
- 4 Handling these callbacks correctly is what separates an app that survives interruption from one that crashes, leaks memory, or loses the user's work.

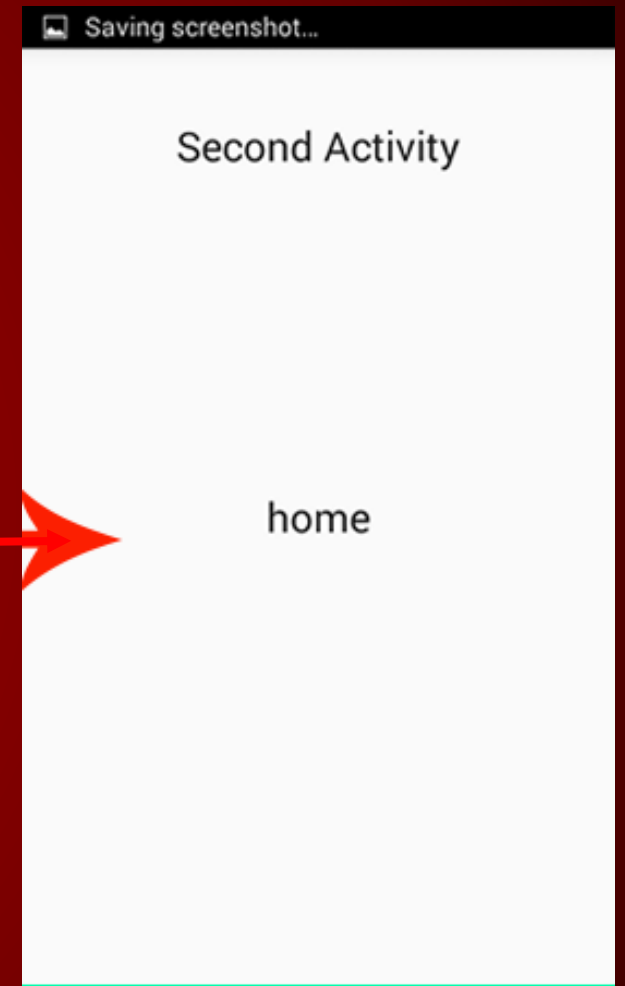
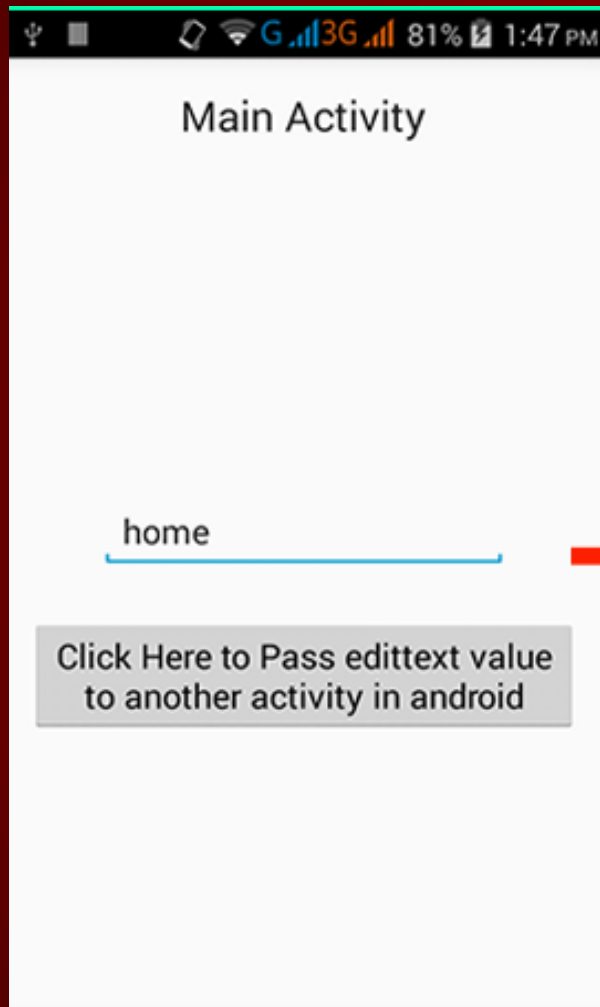
*Coming up — the six states, in order:*



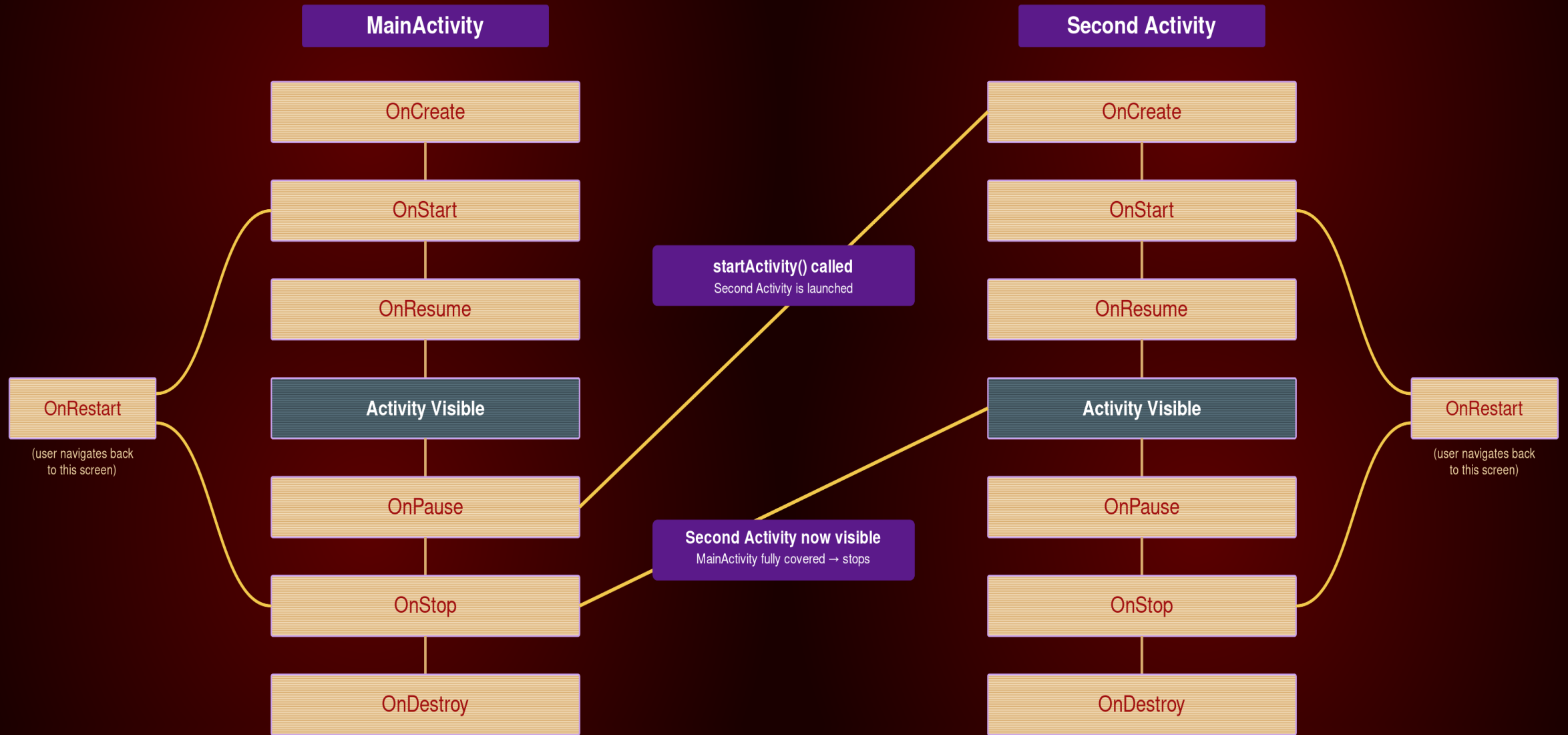
# Activity Life Cycle Methods

- onCreate
- onStart
- onResume
- onRestart
- onPause
- onStop
- onDestroy

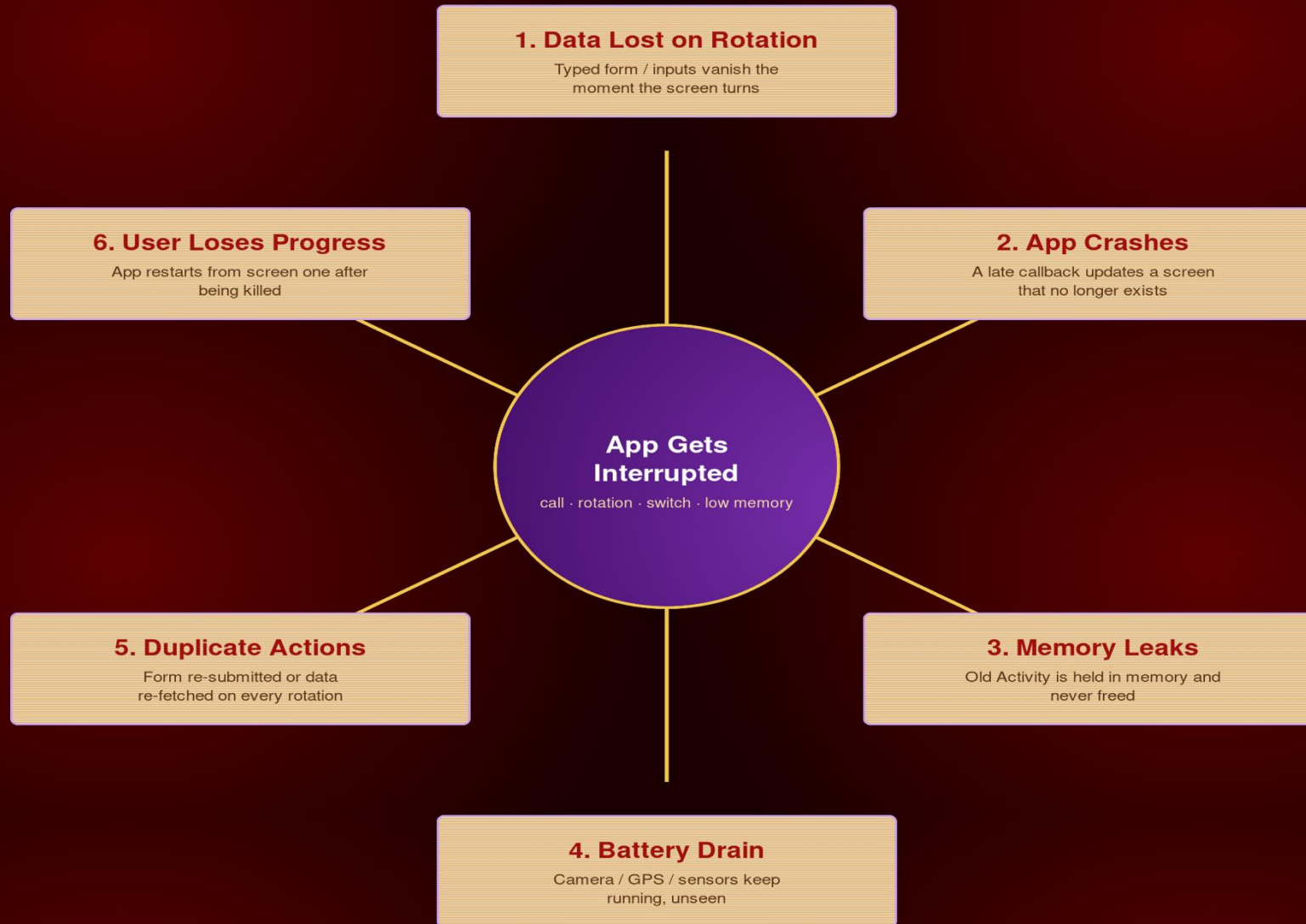
# Activity Life Cycle



# Activity Life Cycle

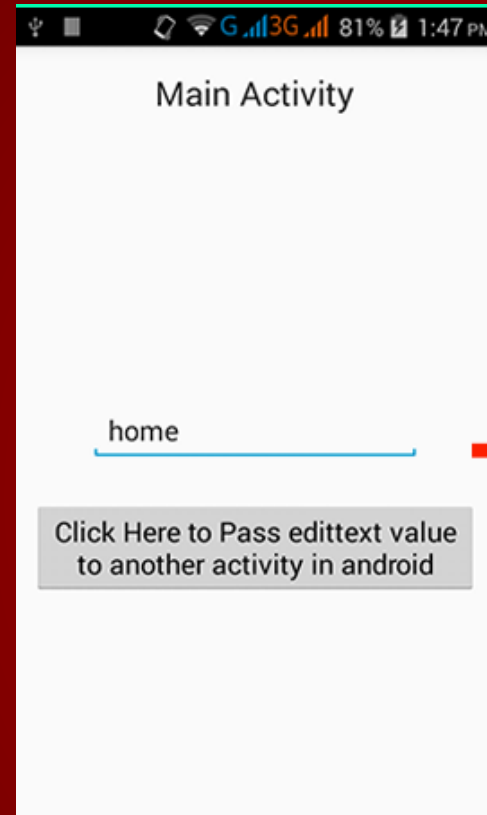
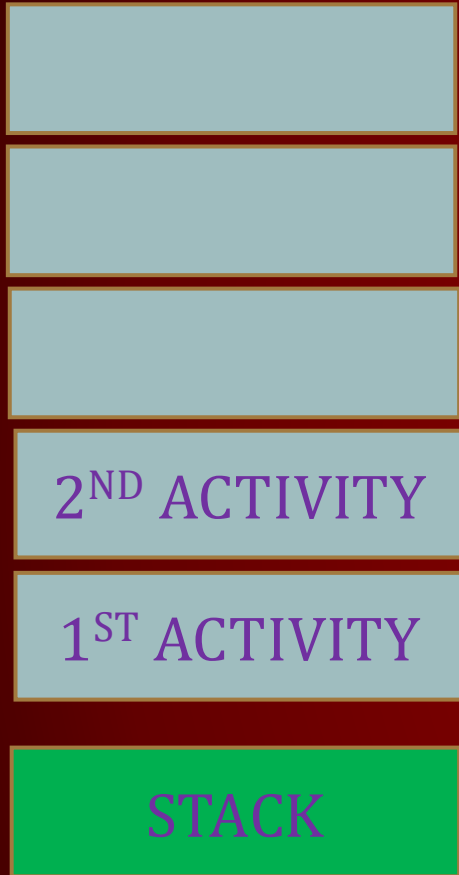


# Ignore the Lifecycle → These Problems Follow



*An app that ignores the lifecycle only works as long as nothing interrupts it — and on a phone, something always does.*

# STACK AND ACTIVITY



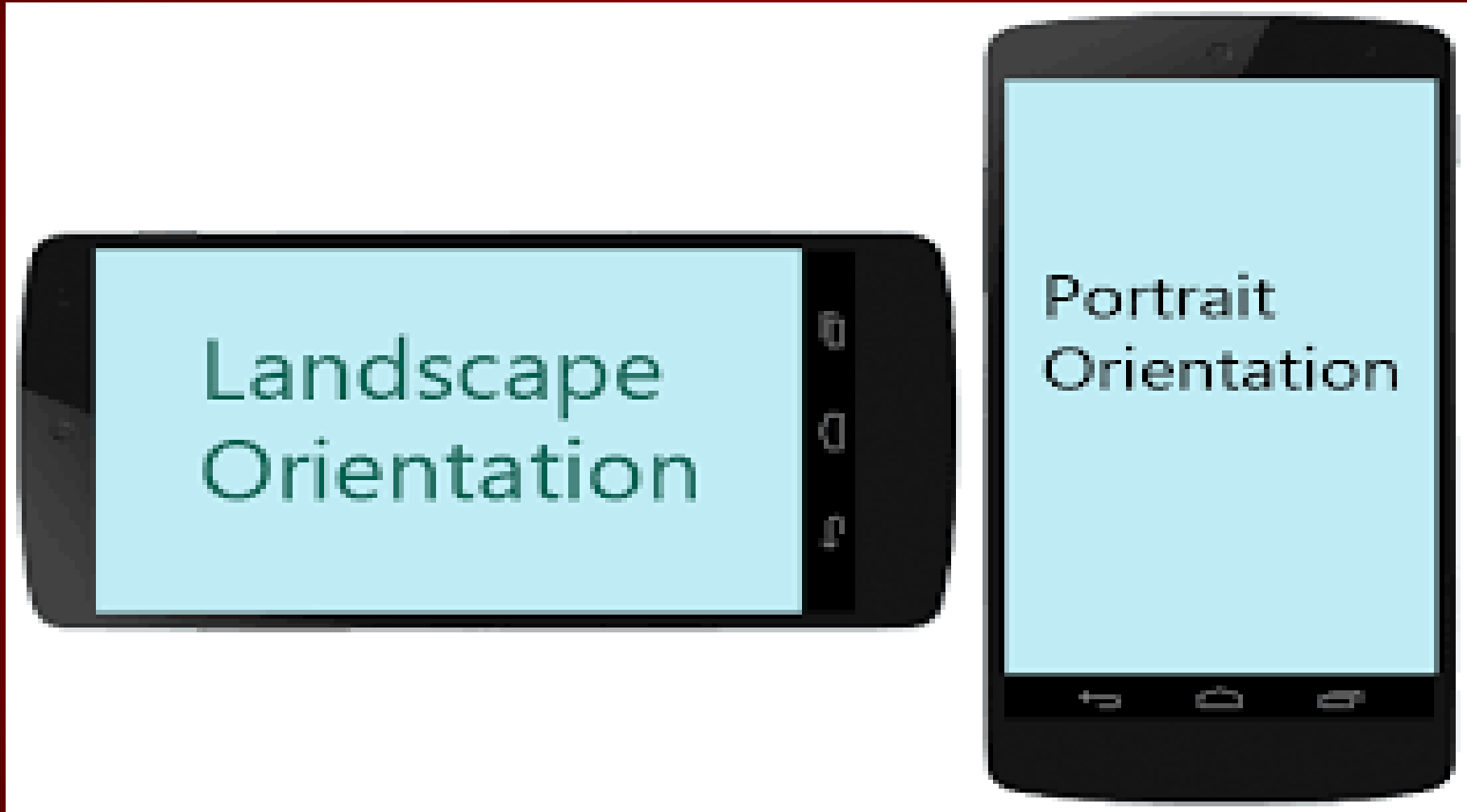
# Activity Life Cycle

|            |                                                                                                                                                                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| onCreate() | Whenever an Activity starts running, the first method to get executed is onCreate(). This method is executed only once during the lifetime. If we have any instance variables in the Activity, the initialization of those variables can be done in this method. After onCreate() method, the onStart() method is executed. |
| onStart()  | During the execution of onStart() method, the Activity is not yet rendered on screen but is about to become visible to the user. In this method, we can perform any operation related to UI components.                                                                                                                     |
| onResume() | When the Activity finally gets rendered on the screen, onResume() method is invoked. At this point, the Activity is in the active state and is interacting with the user.                                                                                                                                                   |
| onPause()  | If the activity loses its focus and is only partially visible to the user, it enters the paused state. During this transition, the onPause() method is invoked. In the onPause() method, we may commit database transactions or perform light-weight processing before the Activity goes to the background.                 |

# Activity Life Cycle

|                          |                                                                                                                                                                                                                                                                             |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>onStop()</code>    | From the active state, if we hit the Home key, the Activity goes to the background and the Home Screen of the device is made visible. During this event, the Activity enters the stopped state. Both <code>onPause()</code> and <code>onStop()</code> methods are executed. |
| <code>onDestroy()</code> | When an activity is destroyed by a user or Android system, <code>onDestroy()</code> function is called.                                                                                                                                                                     |

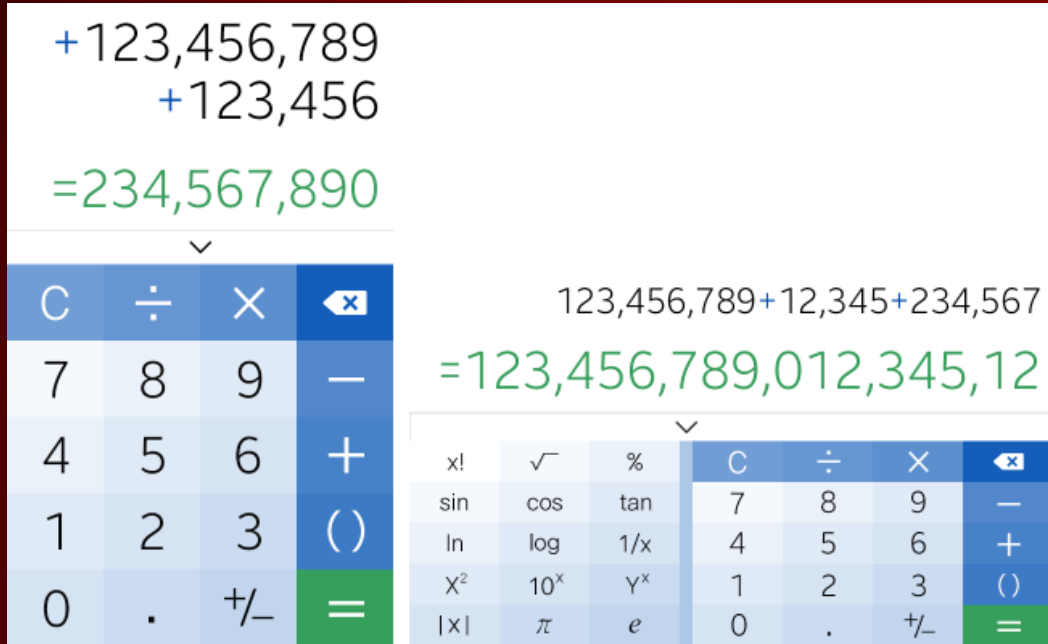
## Screen Orientation and Activity Life cycle



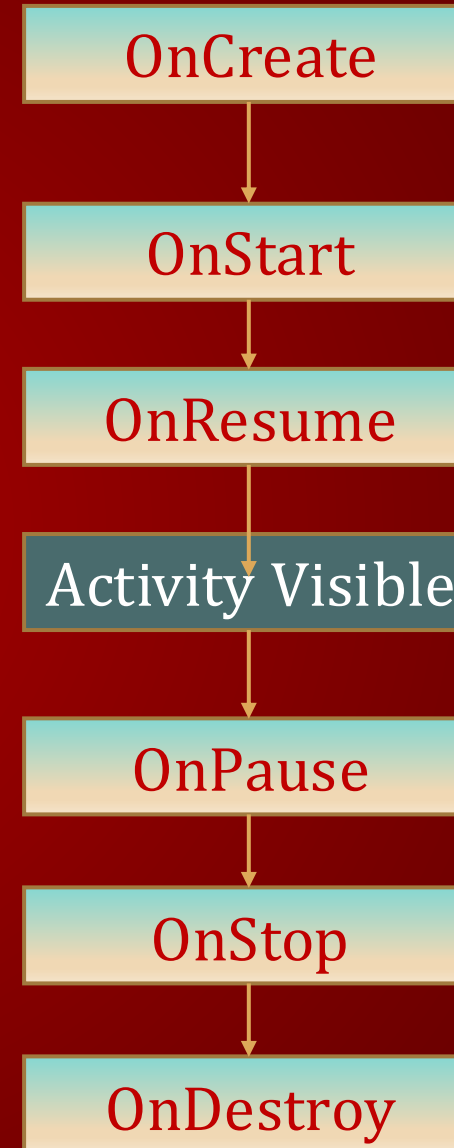
# Screen Orientation and Activity Life cycle

- ❑ On rotation of Screen
  - ❑ Portrait to landscape or vice versa
    - ❑ Activity is destroyed
    - ❑ Activity is recreated fresh in requested orientation

# Consequences of Screen Orientation



- Preserve the state of previous screen orientation
- User should not know that the activity is destroyed



# Solutions to Handle Screen Orientation

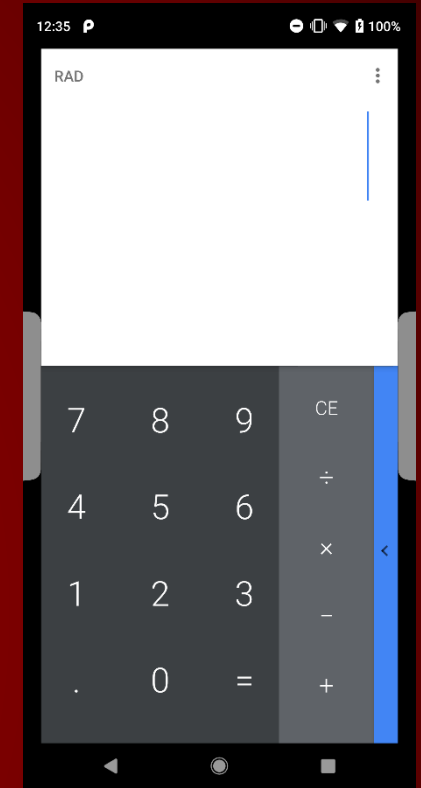
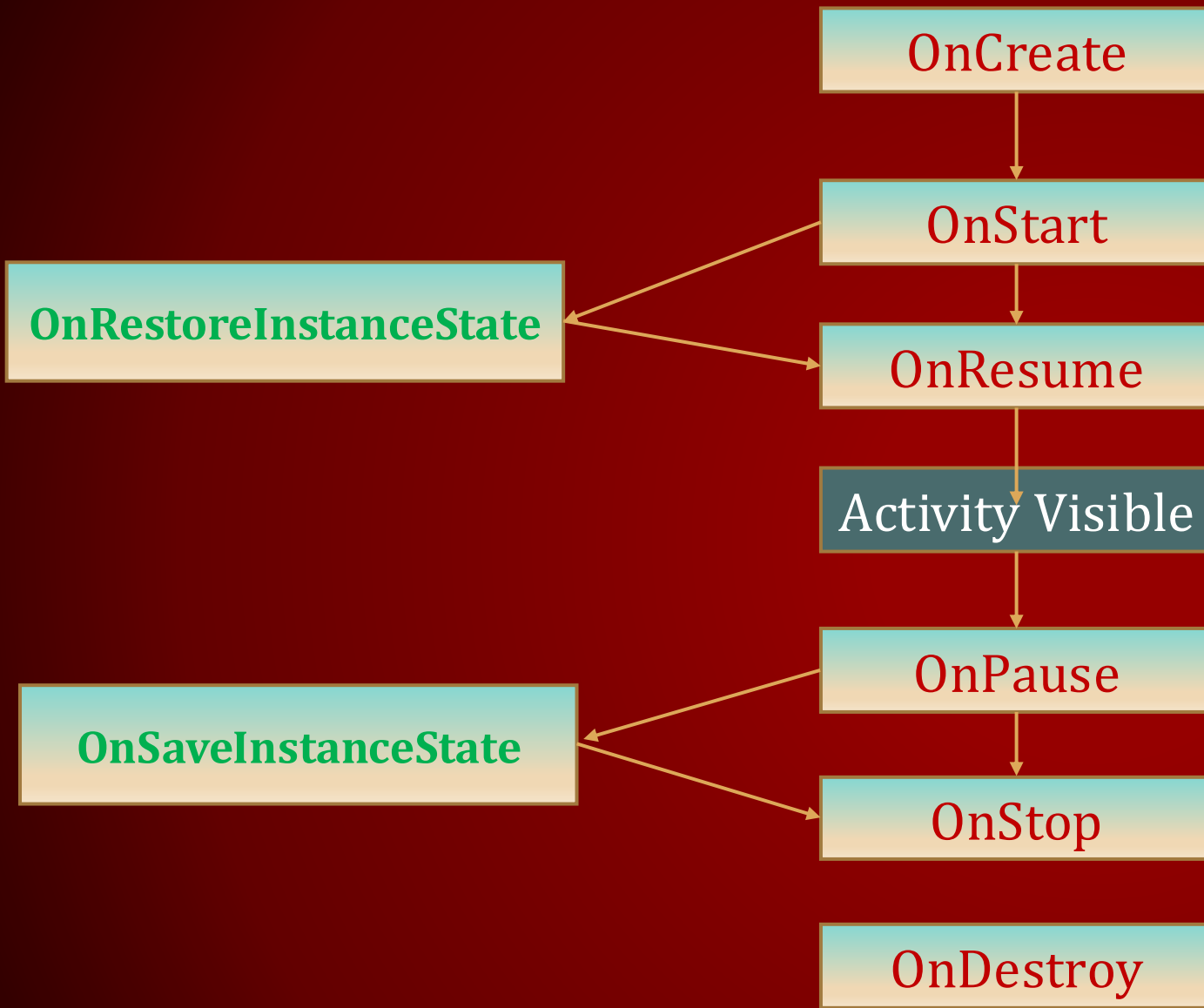
## 1. Restoring the Activity State

- Activity is destroyed and recreated
- Use **onRestoreInstanceState** and **onSaveInstanceState**

## 2. Handling configuration change yourself

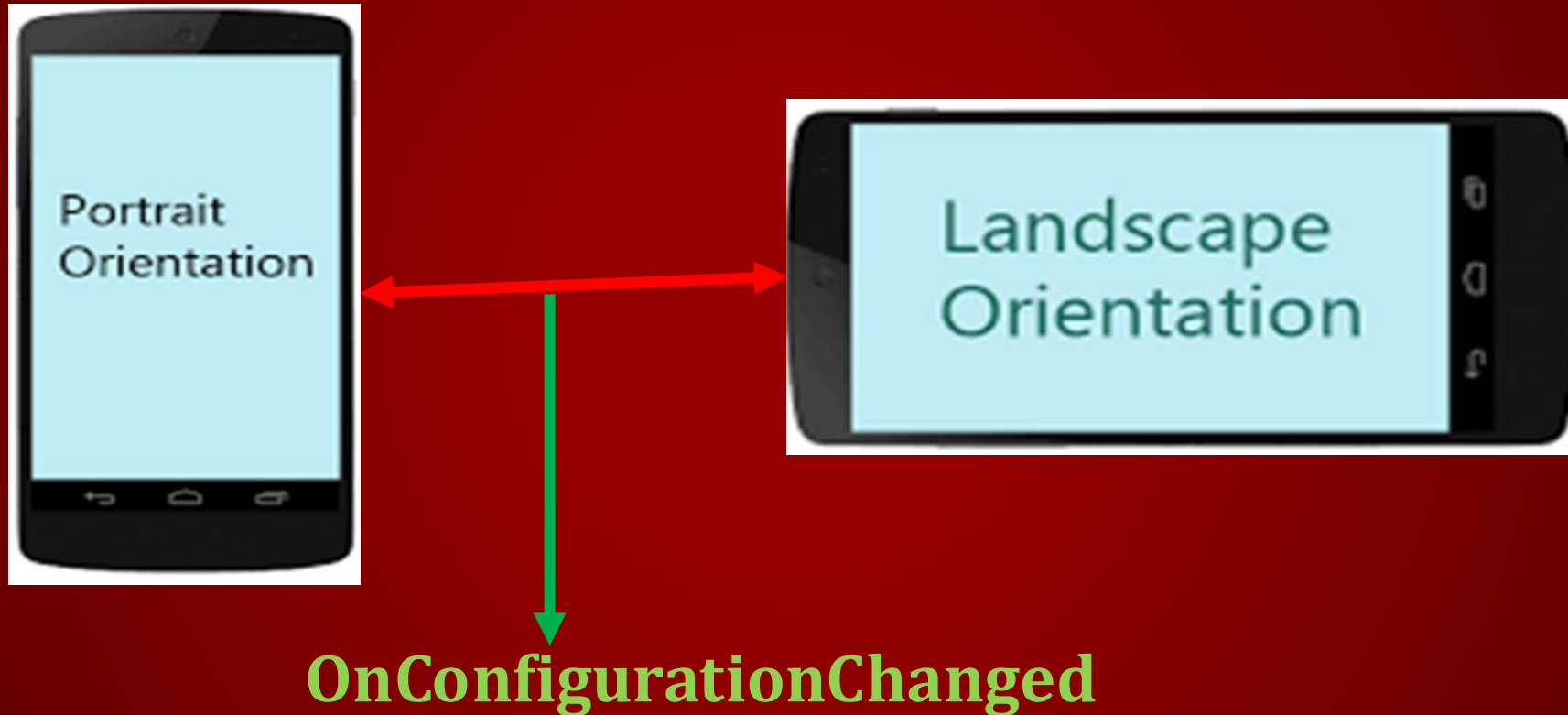
- Activity is not destroyed
- Use **onConfigurationChanged**

# Method 1- Solution to Handle Screen Orientation



## Method 2- Solution to Handle Screen Orientation

### ❑ Handling Configuration Change



- Declare android:configchanged attribute in Manifest file under <activity>
- Override onConfigurationChanged() in Activity class.

# Summary- Solutions to Handle Screen Orientation

## ❑ Method1

- ❖ Is generally preferred and recommended

## ❑ Against GuideLines

- ❖ Not Recommended
- ❖ Should be used only when restoring activity back is expensive and slow